LSGI3245 – Geospatial Database Management and Design

Lecture 6 - Database Design

Instructor: Yue YU

Department of Land Surveying and Geospatial Science,

The Hong Kong Polytechnic University

Tentative Schedule

Week	Lecture (Tuesday)		Lab (Wednesday)	
1	1-Sept	Course Introduction	Lab Introduction and Software Installation	2-Sept
2	8-Sept	Relational Model	Introduction to PostgreSQL (Part 1)	9-Sept
3	15-Sept	SQL (Part 1)	Introduction to PostgreSQL (Part 2)	16-Sept
4	22-Sept	SQL (Part 2) Release Information for Final Project	Query and Data Manipulation Using SQL (Part 1)	23-Sept
5	29-Sept	Intermediate SQL	Query and Data Manipulation Using SQL (Part 2)	30-Sept
6	6-Oct	Database Design	Introduction to PostGIS	7-Oct
7	13-Oct	Spatial Database & PostGIS	PostGIS: Spatial Datatype and Relationships	14-Oct
8	20-Oct	Test 1	PostGIS: Spatial Queries	21-Oct
9	27-Oct	PostGIS & Spatial Queries	Students Work on Final Projects (Part 1)	28-Oct
10	3-Nov	Spatial Storage and Indexing	Students Work on Final Projects (Part 2)	4-Nov
11	10-Nov	Test 2	Students Compile Final Project Reports (Part 1)	11-Nov
12	17-Nov	Project Presentation (Part 1)	Students Compile Final Project Reports (Part 2)	18-Nov
13	24-Nov	Project Presentation (Part 2)	Summary and Outlook	25-Nov

Phase Test 1

Test Duration & Instructions



- Duration: **60 minutes** (***9:00 – 10:00 AM***)
- Instructions:
  - You will be provided with **a two-page exam paper** and **two blank answer sheets**.
  - Write down your **student ID and name** on BOTH the exam paper and the answer sheets.
  - **Answer ALL questions** from Section A and B. (total questions = 8, total marks = 100)
  - **Return the exam paper and answer sheets** to the instructor at the end of the exam.

Test Duration & Instructions



- Duration: **60 minutes** (***9:00 – 10:00 AM***)
- Format: **open-book** test
- Scope: **Lecture 1 - Lecture 6**

Warning: No Electronic Devices

Recap



- Understand different types of **join** expressions.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

- Understand the concept of ***view***.

Learning Outcomes of Today's Class

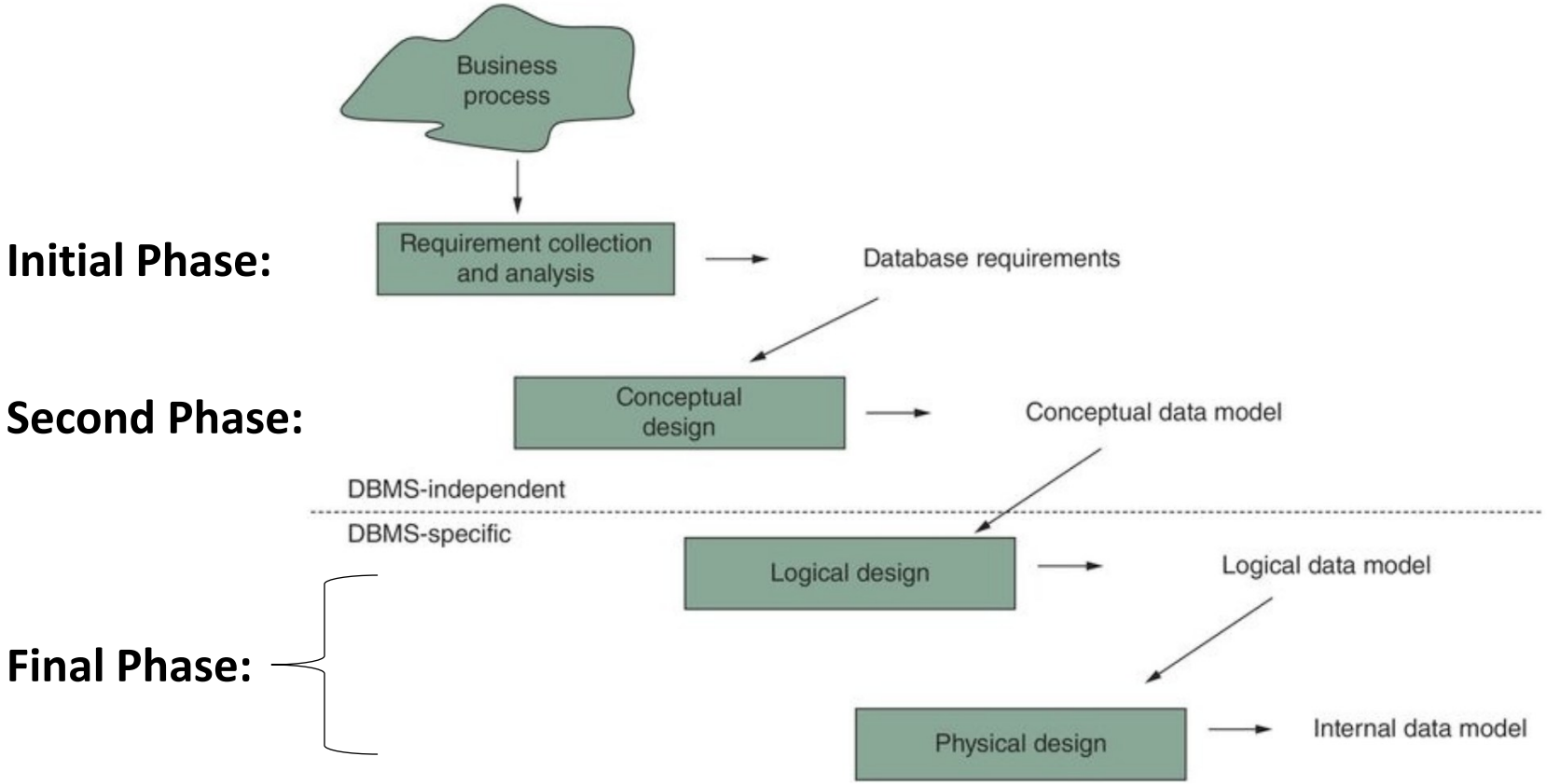


- Understand the process of database design.
- Understand the usage of ***ER diagram***.
- Understand how to map ***ER diagrams*** to ***relation schemas***.
- Understand the concept of **normalization**.

Database Design

Design Phases

- Designing a database is a multi-step process



Source: https://ekonomika.be

Design Phases (Cont.)

- Initial phase (**requirement collection and analysis**)
  - Characterize fully the data needs of the prospective database users.
  - Various techniques can be used, such as interviews or surveys with end-users, inspections of the documents, etc.
  - The outcome of this phase is a specification of user requirements.

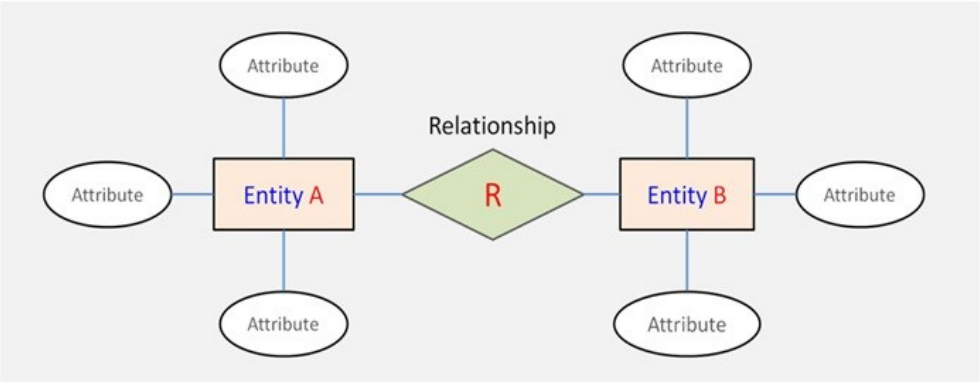




Source: Google Image

Design Phases (Cont.)

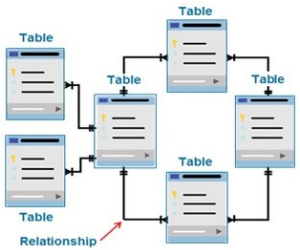
- Second phase (**conceptual design**) – choosing a data model
  - Applying the concepts of the chosen data model and translating user requirements into a conceptual schema of the database.
  - The chosen data model should be both easy to understand for the business user and formal enough for the database designer who will use it in the next step.
  - The **entity-relationship (ER) model** is typically used to represent the conceptual design.



Source: Google Image

Design Phases (Cont.)

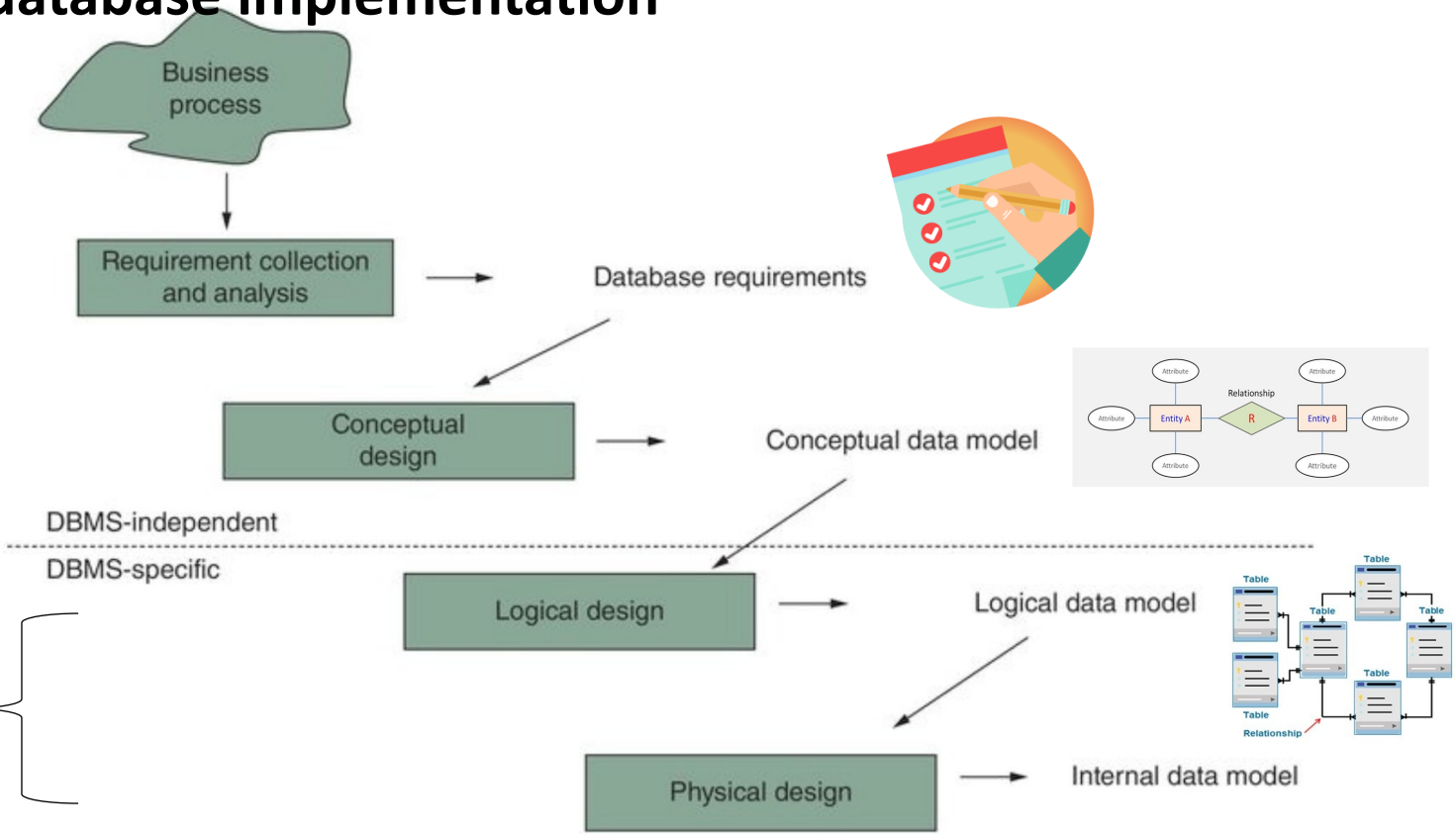
- Final Phase (**database implementation**) – moving from an abstract data model to the implementation of the database
  - **Logical Design** – Deciding on the database schema.
    - The designers map the high-level conceptual schema onto the implementation data model of the database system that will be used.



    - The implementation data model is typically the **relational data model**, and this step typically consists of mapping the conceptual schema defined using the **ER model** into a **relation schema**.
  - **Physical Design** – Deciding on the physical layout of the database
    - Physical features of the database are specified, including the form of file organization and choice of index structures.

Summary of Design Phases

- A multi-step process: **requirement collection → conceptual design → database implementation**

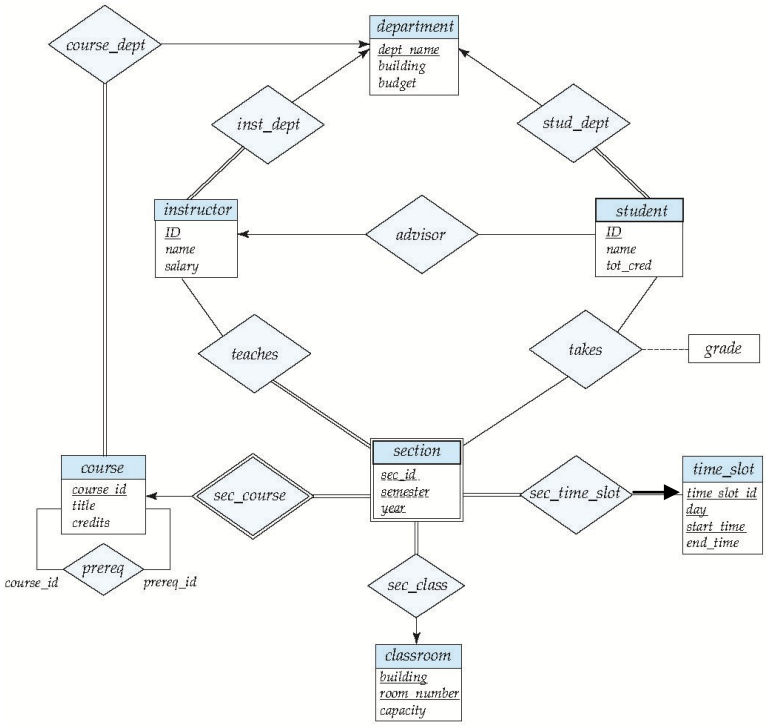


Source: https://ekonomika.be

The Entity-Relationship (ER) Model

The Entity-Relationship (ER) Model

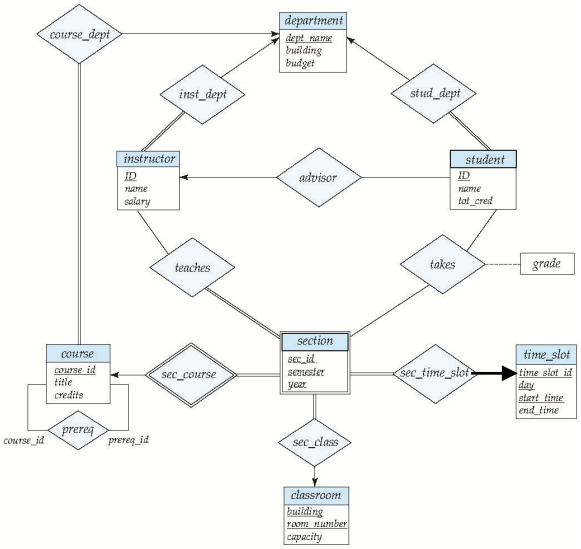
- The entity relationship (ER) model was introduced and formalized by Peter Chen in 1976. It is one of the most popular data models for **conceptual data modeling**.
- The ER data model employs three basic concepts:
  - Entity sets
  - Relationship sets
  - Attributes
- The ER model also has an associated diagrammatic representation, the **ER diagram**, which can express the overall logical structure of a database graphically.



Source: http://www.db-book.com/

Basic Structure of the ER Diagram

- **Rectangles** divided into two parts represent entity sets. The first part (shaded in blue), contains the name of the entity set. The second part contains the names of all attributes of the entity set.
- **Diamonds** represent relationship sets.
- **Undivided rectangles** represent the attributes of a relationship set.
- **Lines** link entity sets to the relationship sets.
- **Dashed lines** link attributes of a relationship set to the relationship set.



Entity Sets

- An **entity** is an object that exists and is distinguishable from other objects.
  - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
- An entity is represented by a set of **attributes**; i.e., descriptive properties possessed by all members of an entity set.
  - Example: instructor = (ID, name, department, salary)
- A subset of the attributes form a **primary key** of the entity set, i.e., uniquely identifying each member of the set.

Entity Sets - Examples

instructor_ID	instructor_name
76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

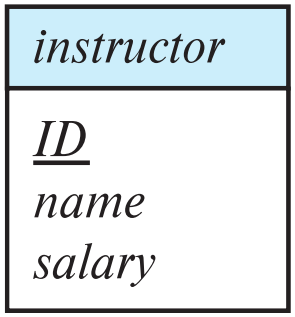
instructor

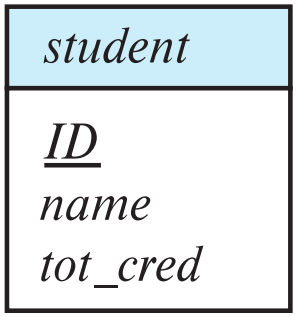
student_ID	student_name
98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Entity Sets in ER Diagram

- Entities can be represented graphically as follows:
  - Rectangles represent **entity sets**.
  - **Attributes** are listed inside entity rectangle.
  - Underline indicates **primary key** attributes.



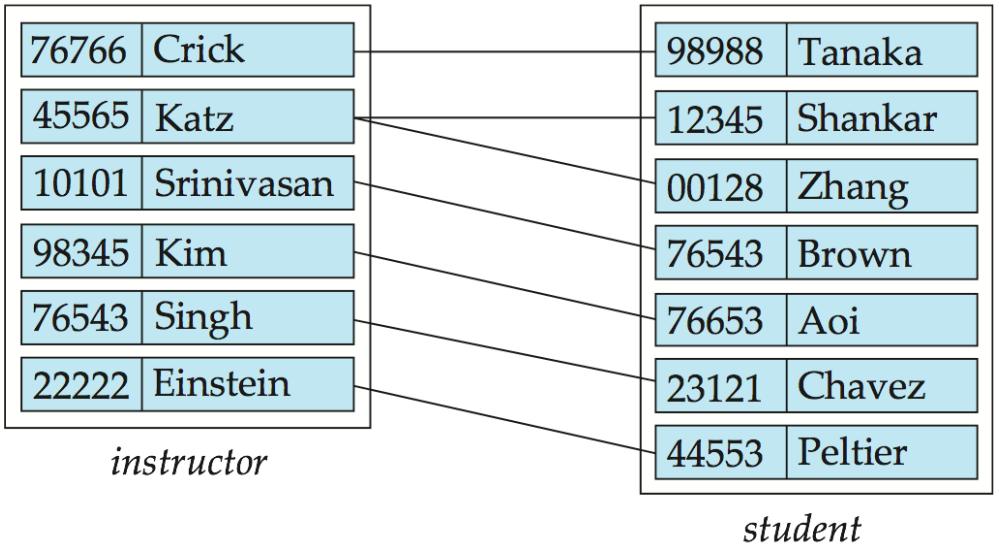


Relationship Sets

- A **relationship** is an association among several entities
- A **relationship set** is a mathematical relation among n>=2 entities, each taken from entity sets

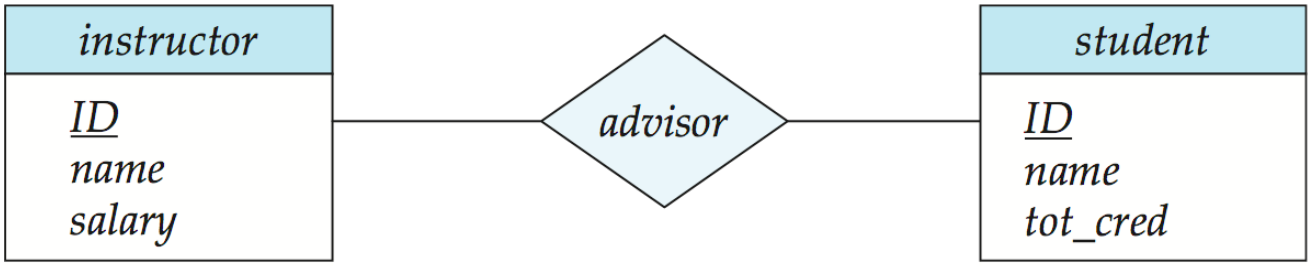
$$\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

where $(e_1, e_2, \ldots e_n)$ is a relationship



Relationship Sets in ER Diagram

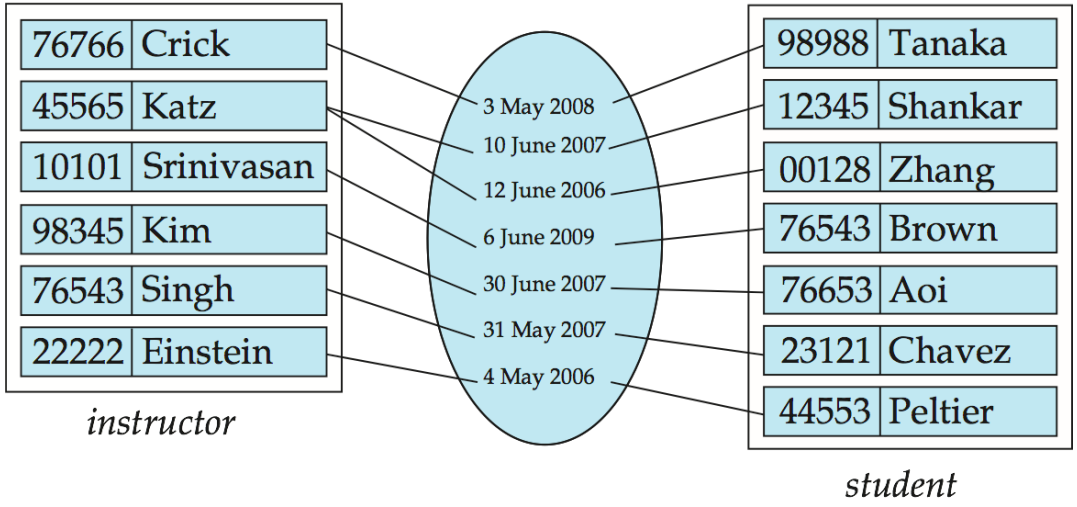
- **Diamonds** represent relationship sets.
- **Lines** link entity sets to the relationship sets.



Source: http://www.db-book.com/

Relationship Sets (Cont.)

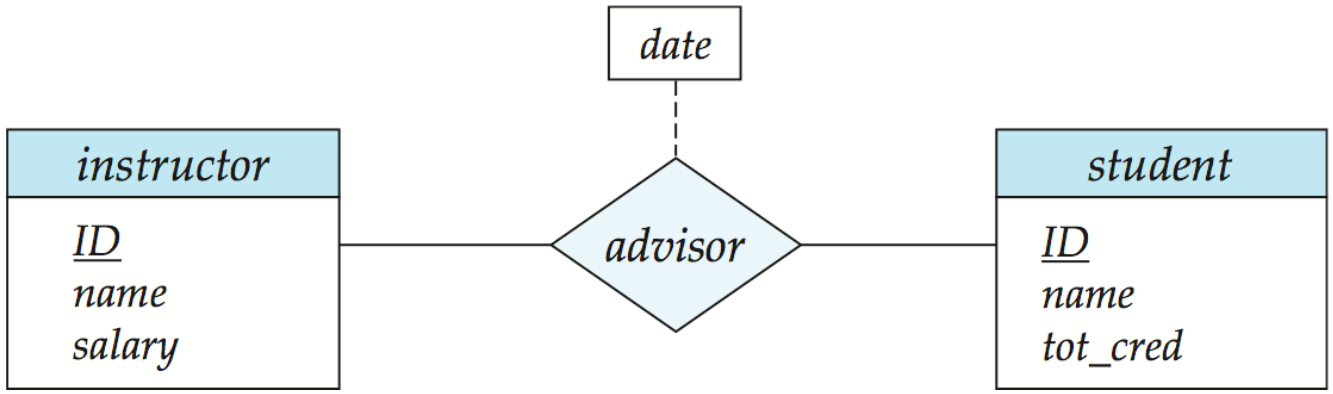
- A relationship may also have attributes called **descriptive attributes**.
- For instance, the ***advisor*** relationship set between entity sets *instructor* and *student* may have the attribute ***date*** which tracks when the student started being associated with the advisor.



Source: http://www.db-book.com/

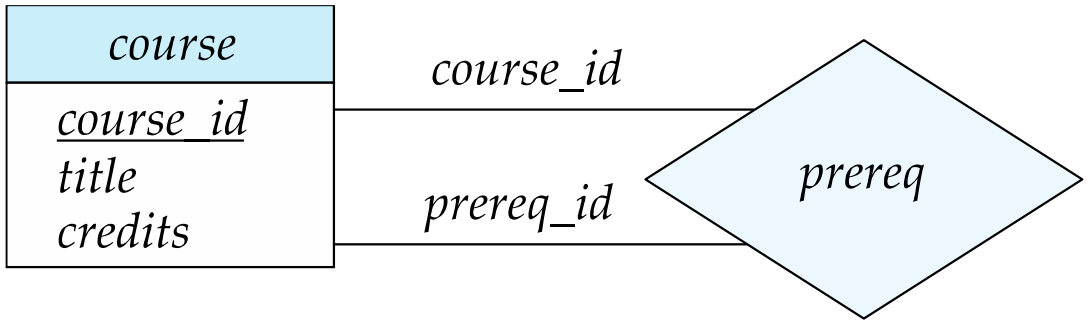
Relationship Sets with Attributes in ER Diagram

- **Undivided rectangles** represent the attributes of a relationship set.
- **Dashed lines** link attributes of a relationship set to the relationship set.



Roles

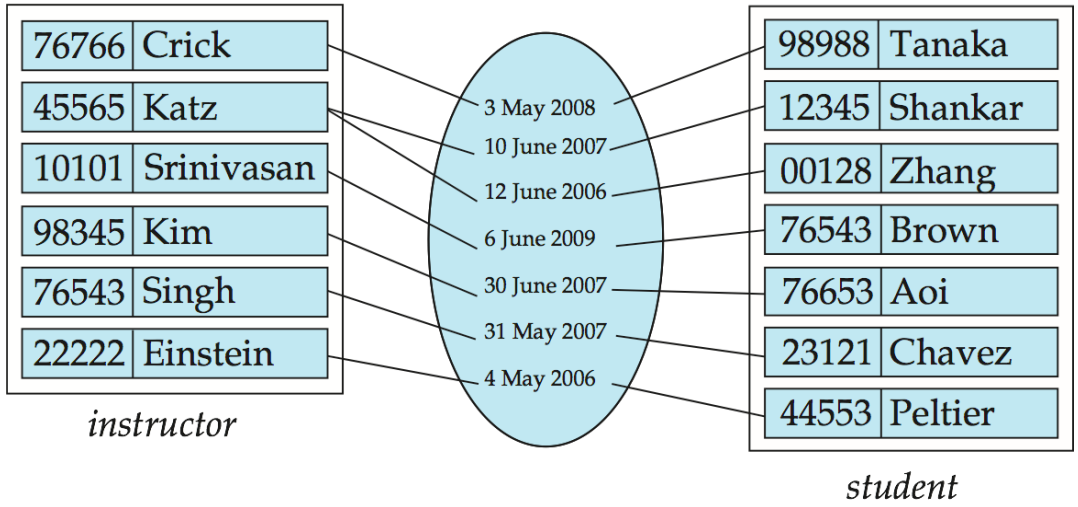
- Entity sets of a relationship need not be distinct
  - Each occurrence of an entity set plays a “role” in the relationship
- Example: The labels “*course_id*” and “*prereq_id*” are called **roles**.



Source: http://www.db-book.com/

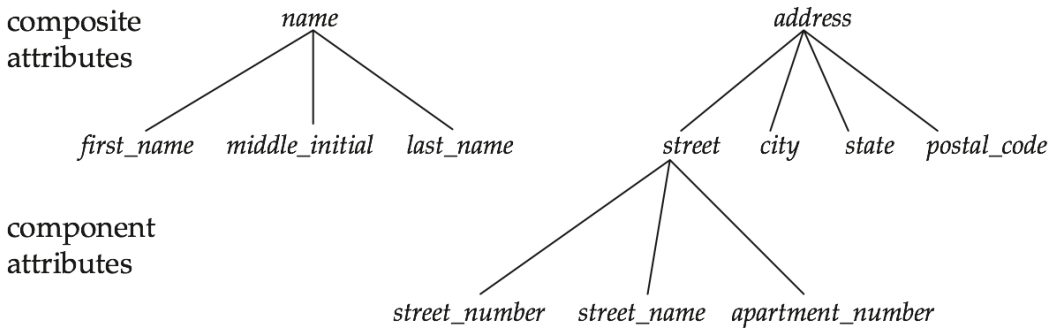
Degree of a Relationship Set

- **Binary** relationship
  - Involve two entity sets (degree = 2).
  - Most relationship sets in a database system are binary.



Attributes

- For each attribute, there is a set of permitted values, called the **domain**, or **value set** of that attribute.
  - Example: The domain of attribute *semester* might be strings from the set {Fall, Winter, Spring, Summer}.
- Common attribute types:
  - Simple and composite attributes.

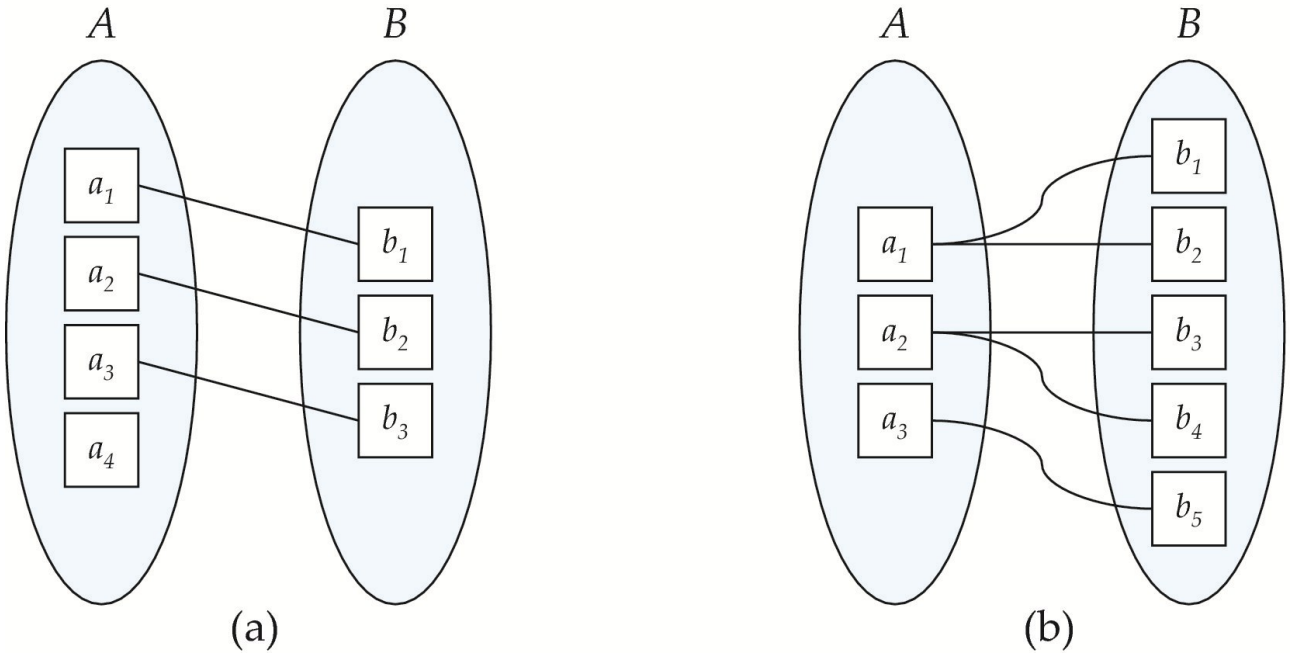


  - Single-valued and multivalued attributes (Example: phone number of *instructor*)

Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set, the mapping cardinality must be one of the following types:
  - One to one
  - One to many
  - Many to one
  - Many to many

Mapping Cardinalities

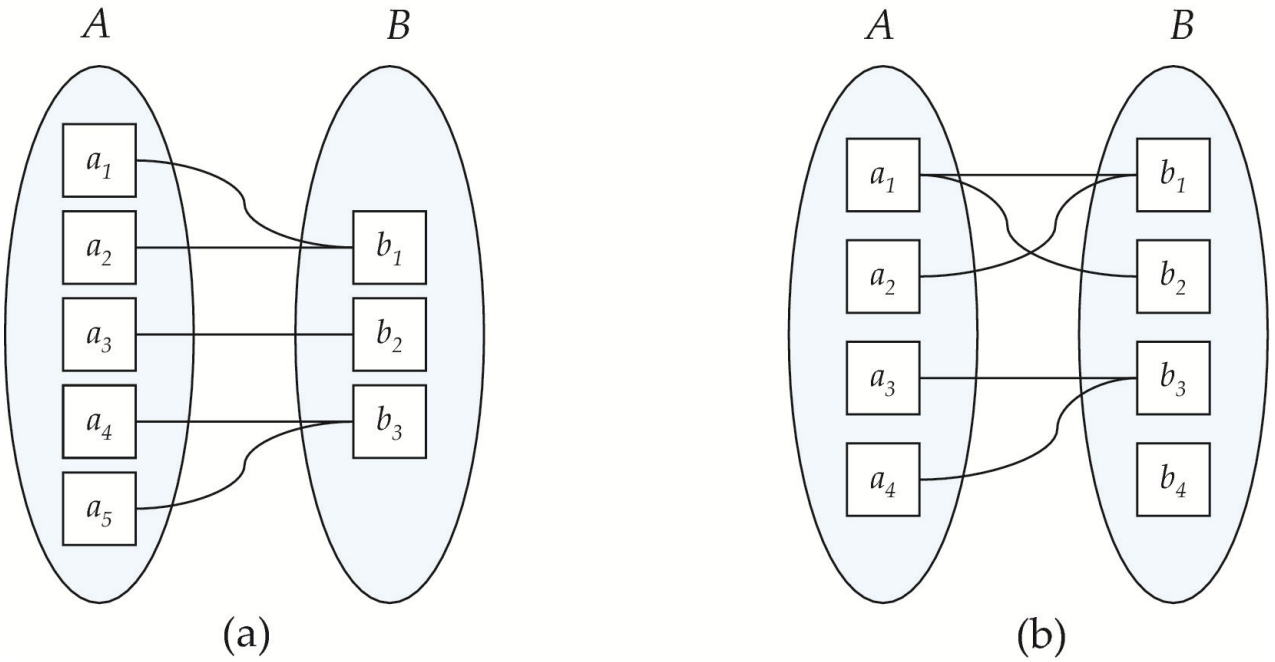


one to one

one to many

Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

Mapping Cardinalities (Cont.)



many to one

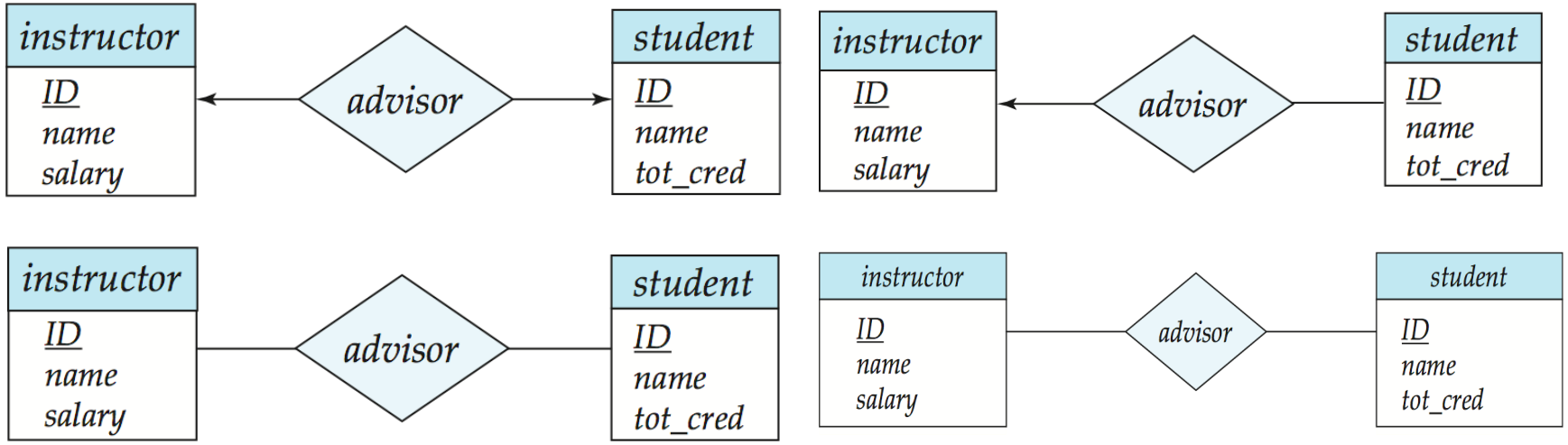
many to many

Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

Representing Cardinality Constraints in ER Diagram

- We express cardinality constraints by drawing either
  - a directed line (→), signifying "one," or
  - an undirected line (—), signifying "many,"

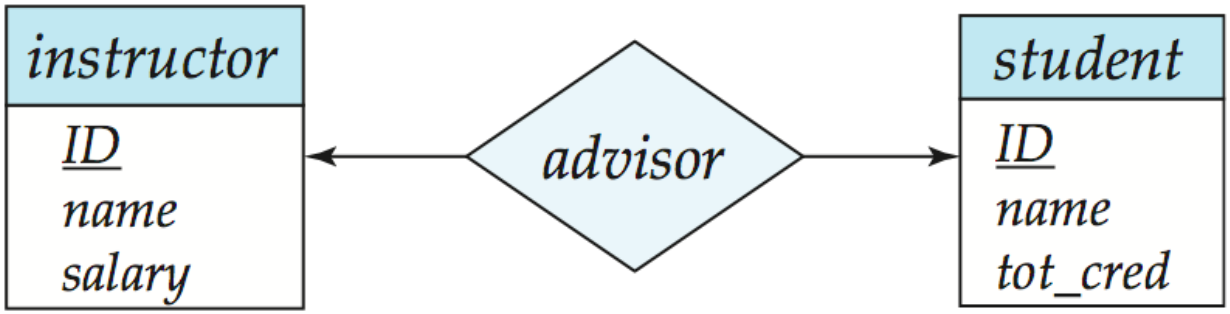
  between the relationship set and the entity set.



Source: http://www.db-book.com/

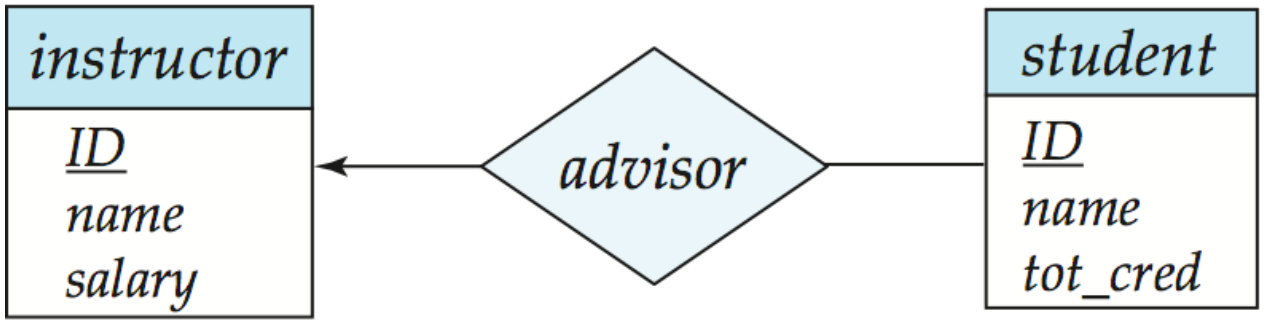
One-to-One Relationship

- One-to-one relationship between an *instructor* and a *student*:
  - An instructor is associated with at most one student via the relationship advisor
  - A student is associated with at most one instructor via advisor



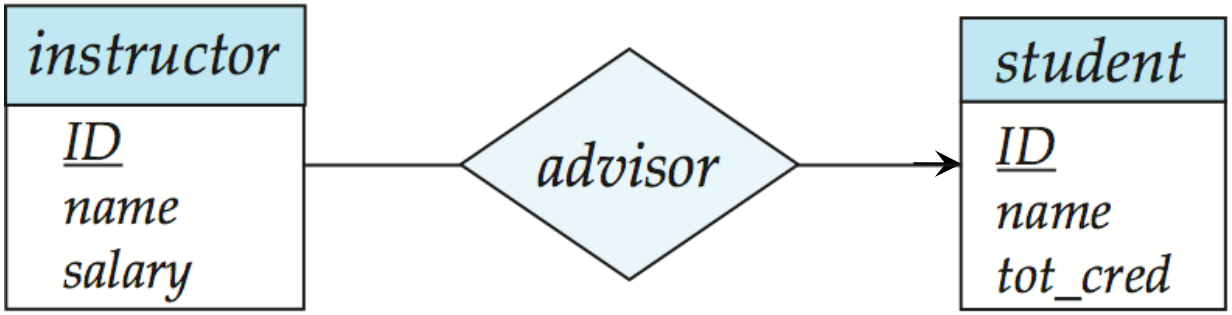
One-to-Many Relationship

- One-to-many relationship between an *instructor* and a *student:*
  - An instructor is associated with several (including 0) students.
  - A student is associated with at most one instructor via advisor.



Many-to-One Relationship

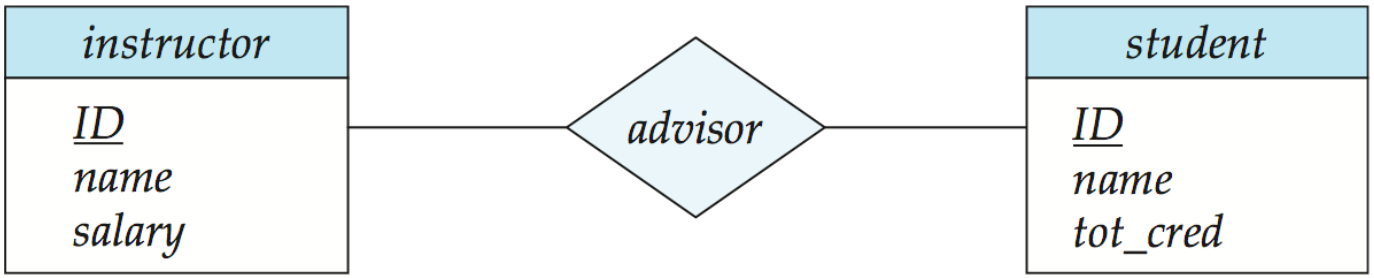
- Many-to-one relationship between an *instructor* and a *student:*
  - An instructor is associated with at most one student via *advisor.*
  - A student is associated with several (including 0) instructors via advisor.



Source: http://www.db-book.com/

Many-to-Many Relationship

- Many-to-many relationship between an *instructor* and a *student:*
  - An instructor is associated with several (possibly 0) student via *advisor.*
  - A student is associated with several (possibly 0) instructors via advisor.



Source: http://www.db-book.com/

Total and Partial Participation

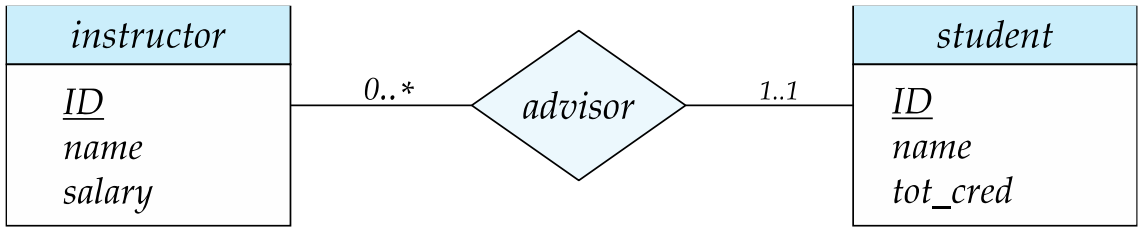
- **Total participation** (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set.
- **Partial participation**: some entities may not participate in any relationship in the relationship set
- Example:



  - Participation of student in *advisor* relation is total (every *student* must have an associated instructor)
  - Participation of instructor in *advisor* is partial

Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality
  - A minimum value of 1 indicates total participation.
  - A maximum value of 1 indicates that the entity participates in at most one relationship
  - A maximum value of * indicates no limit.
- Example:

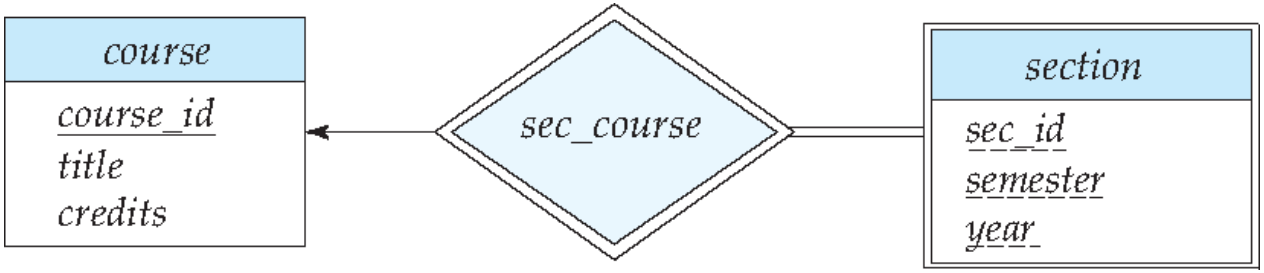


  - An instructor can advise 0 or more students.
  - A student must have 1 advisor, and cannot have multiple advisors (total participation)

Source: http://www.db-book.com/

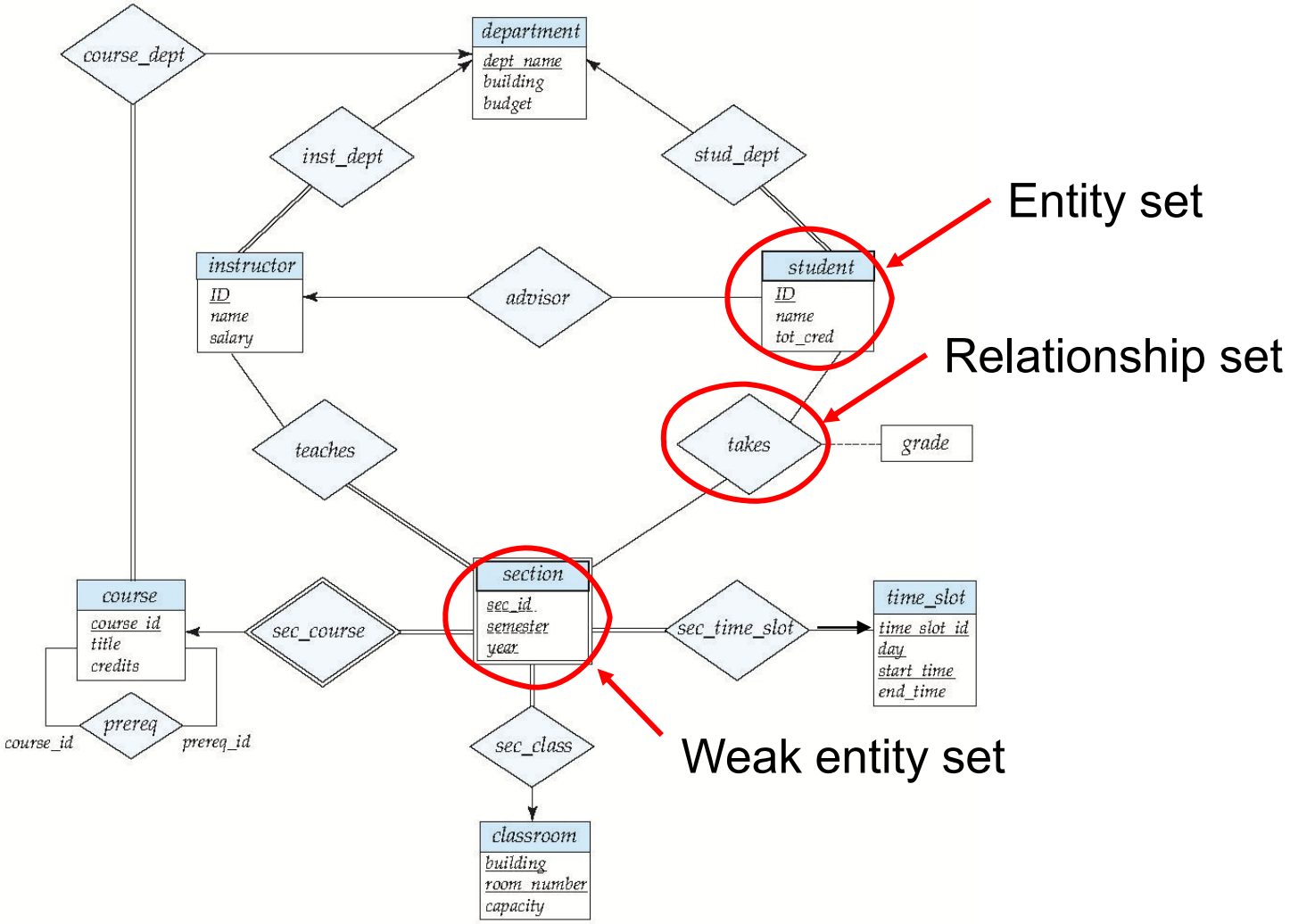
Expressing Weak Entity Sets

- In ER diagrams, a weak entity set is depicted via a double rectangle.

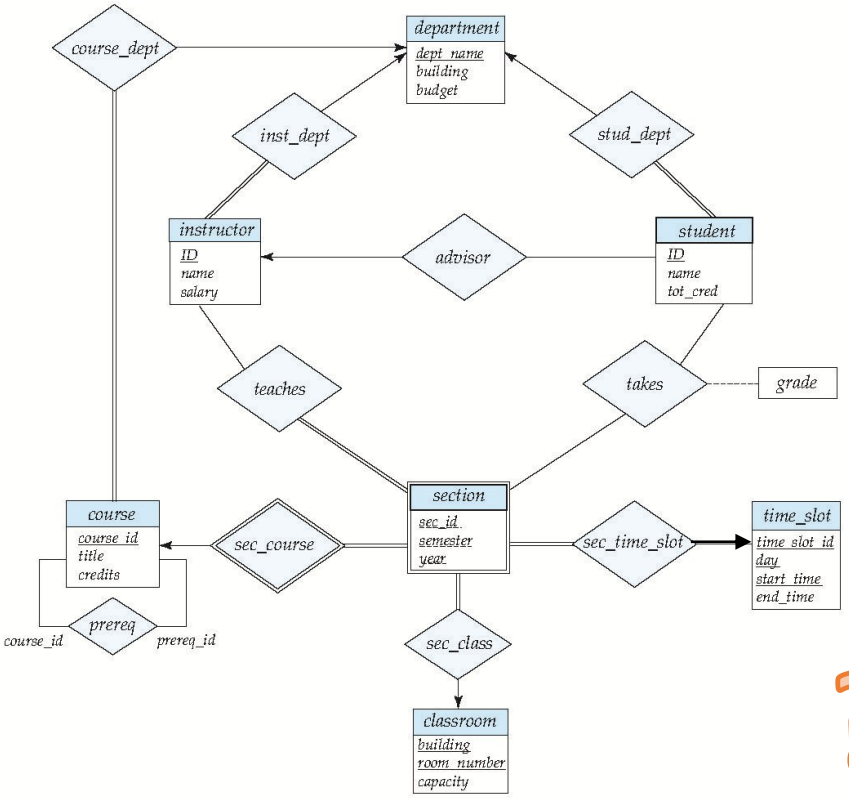


- The information {*section_id, semester, year*} stored in section relation needs to be associated with a *course_id* in order to be distinct.
- An entity set that does not have sufficient attributes to form a primary key is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.

ER Diagram for a University Enterprise



ER Diagram vs. Relational Schema Diagram

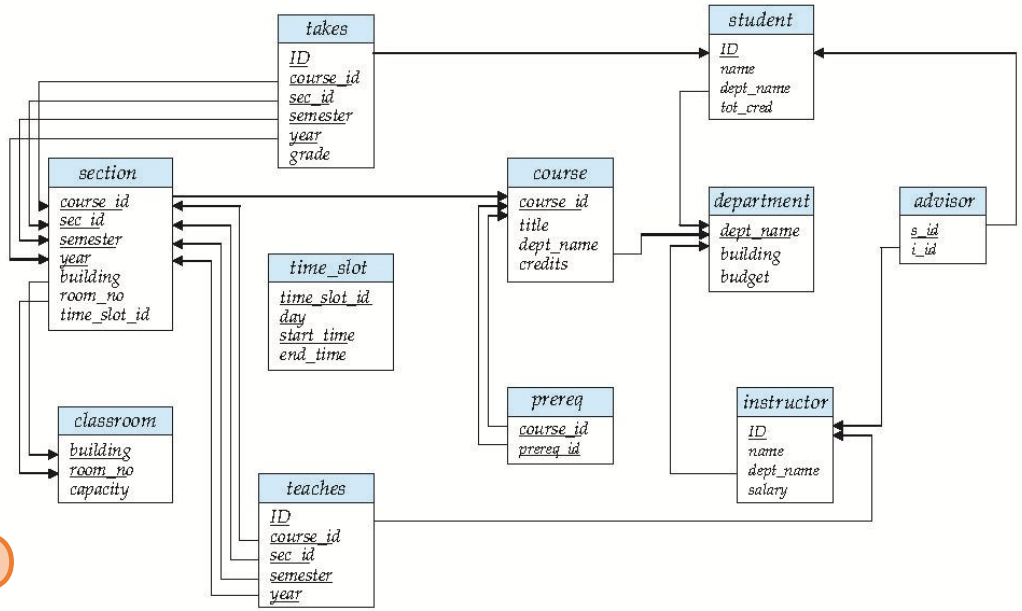


ER Diagram

(Conceptual Data Model)







Relational Schema Diagram

(Logical Data Model)

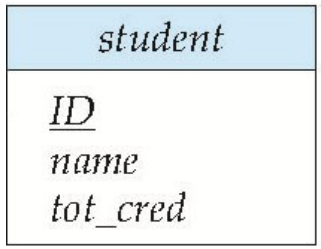
Map ER Diagrams to Relation Schemas

Reduction to Relation Schemas

- Entity sets and relationship sets can be expressed uniformly as relation schemas that represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of schemas.
- For each entity set and relationship set, there is a *unique* schema that is assigned the name of the corresponding entity set or relationship set. However, *redundancy* exists and some schemas can be combined.
- Each schema has a number of columns (generally corresponding to attributes), which have unique names.

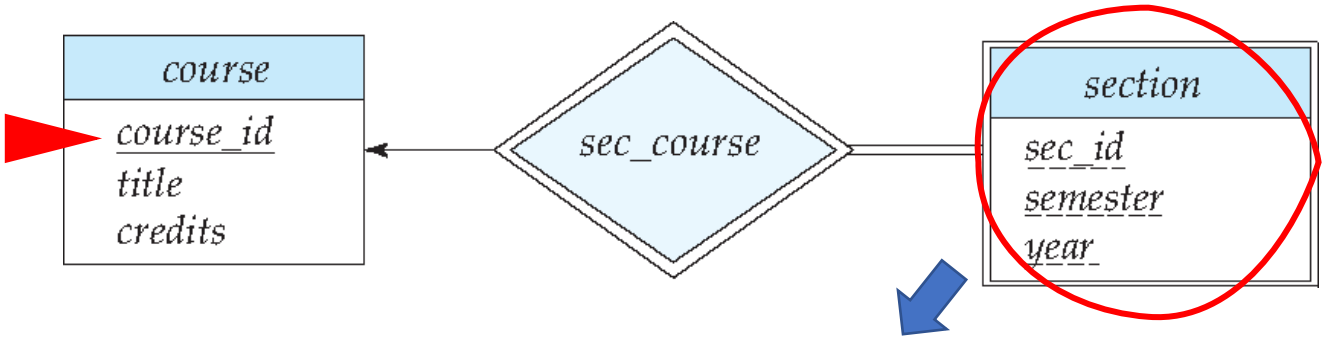
Representing Entity Sets

- A **strong** entity set reduces to a schema with the same attributes



student(ID, name, tot_cred)

- A **weak** entity set becomes a table that includes a column for the primary key of the identifying strong entity set

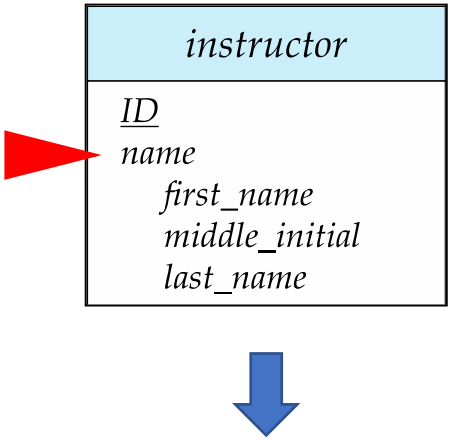


*section (**course_id**, sec_id, sem, year)*

Source: http://www.db-book.com/

Representing Entity Sets (Cont.)

- **Composite attributes** are flattened out by creating a separate attribute for each component attribute
  - Example: given entity set ***instructor*** with composite attribute ***name*** with component attributes ***first_name, middle_initial,*** and ***last_name***

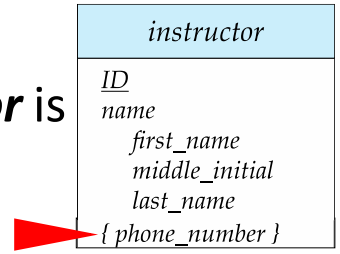


instructor(<u>*ID*</u>*,* ***first_name, middle_initial, last_name****)*

Source: http://www.db-book.com/

Representing Entity Sets (Cont.)

- A **multivalued attribute** ***M*** of an entity ***E*** is represented by a separate schema ***EM***. Schema ***EM*** has attributes corresponding to the primary key of ***E*** and an attribute corresponding to multivalued attribute ***M***.
  - Example: Multivalued attribute ***phone_number*** of ***instructor*** is represented by a schema ***inst_phone***:



$$inst_phone = (\underline{ID}, \underline{\textbf{phone_number}})$$

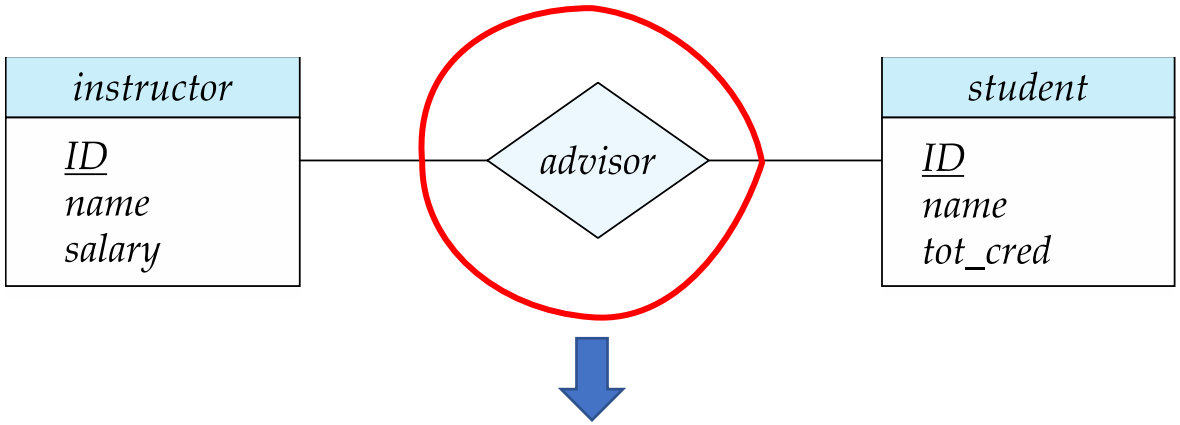
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema ***EM***.
  - For example, an ***instructor*** entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples: (22222, 456-7890) and (22222, 123-4567)

ID	phone_number
22222	456-7890
22222	123-4567

Source: http://www.db-book.com/

Representing Relationship Sets

- A **many-to-many** relationship set is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
  - Example: schema for relationship set ***advisor***

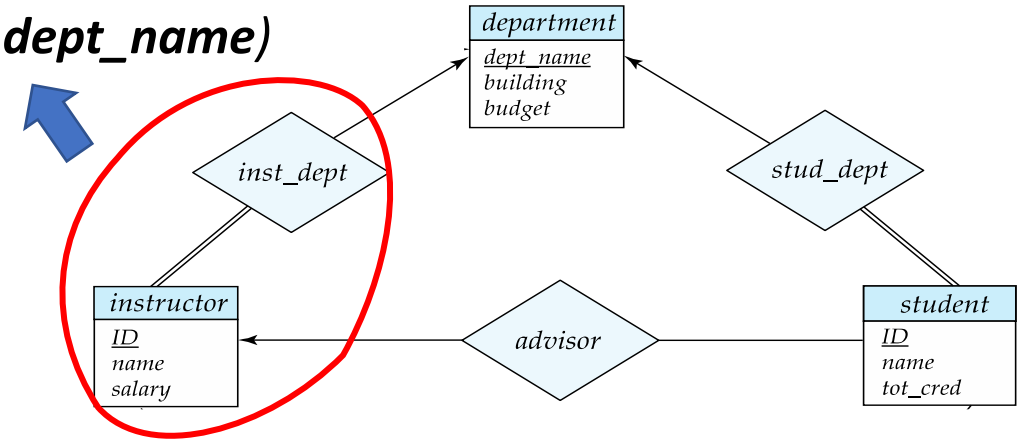


Source: http://www.db-book.com/

Representing Relationship Sets (Cont.)

- **Many-to-one** and **one-to-many** relationship sets that are **total** on the many-side can be represented by adding an extra attribute to the "many" side, containing the primary key of the "one" side
  - Example: Instead of creating a schema for relationship set ***inst_dept***, add an attribute ***dept_name*** to the schema arising from entity set ***instructor***

*instructor(ID, name, salary, **dept_name**)*

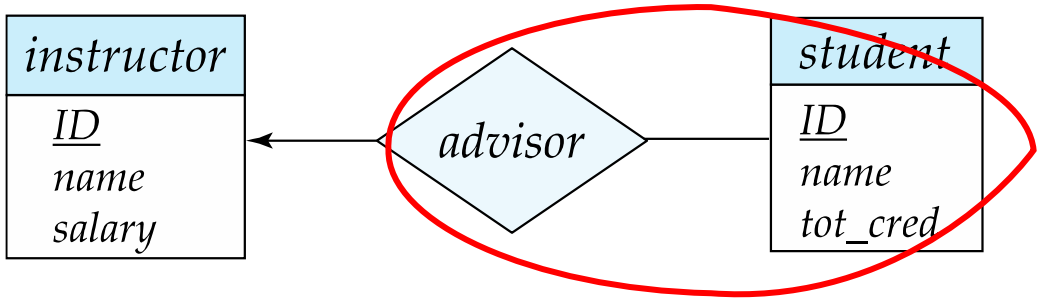


Source: http://www.db-book.com/

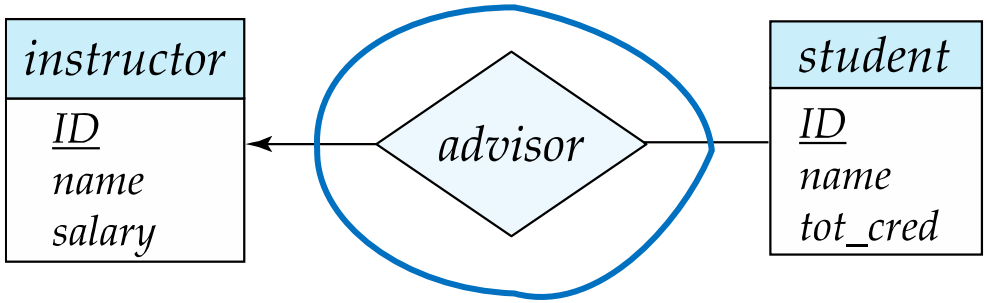
Representing Relationship Sets (Cont.)

- If participation is **partial** on the "many" side, replacing a schema by an extra attribute in the schema corresponding to the "many" side could result in **null** values

*student(ID, name, tot_cred, **advisor_id**)*



Can be **null**



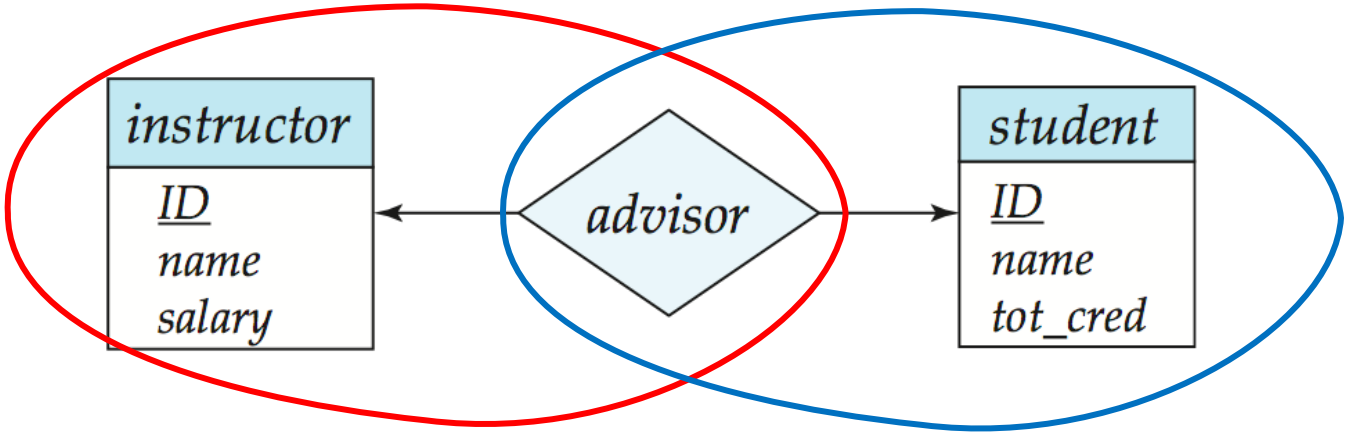
advisor *= (s_id, i_id)*

Source: http://www.db-book.com/

Representing Relationship Sets (Cont.)

- For **one-to-one** relationship sets, either side can be chosen to act as the "many" side
  - That is, an extra attribute can be added to either of the tables corresponding to the two entity sets

*instructor(ID, name, salary, **adv_stud_id**)*



*student(ID, name, tot_cred, **advisor_id**)*

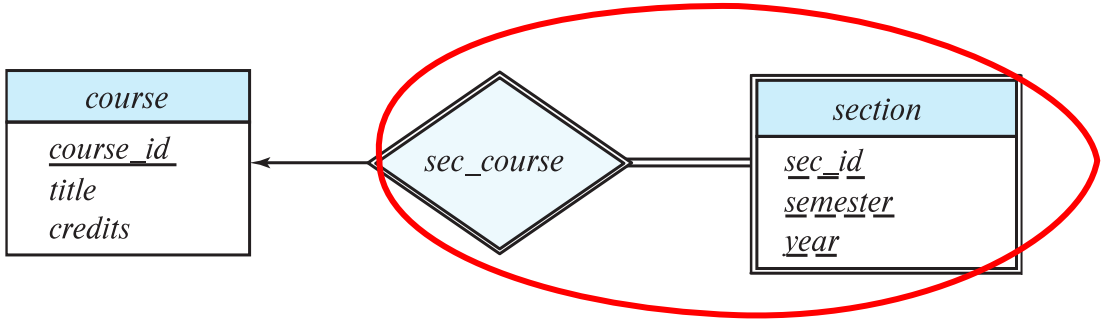
Source: http://www.db-book.com/

Representing Relationship Sets (Cont.)

- The schema corresponding to a relationship set linking a **weak entity set** to its **identifying strong entity set** is redundant.
  - Example: The ***section*** schema already contains the attributes that would appear in the ***sec_course*** schema

section(***course_id***, *sec_id, semester, year*)

Primary key in the identifying strong entity set



Source: http://www.db-book.com/

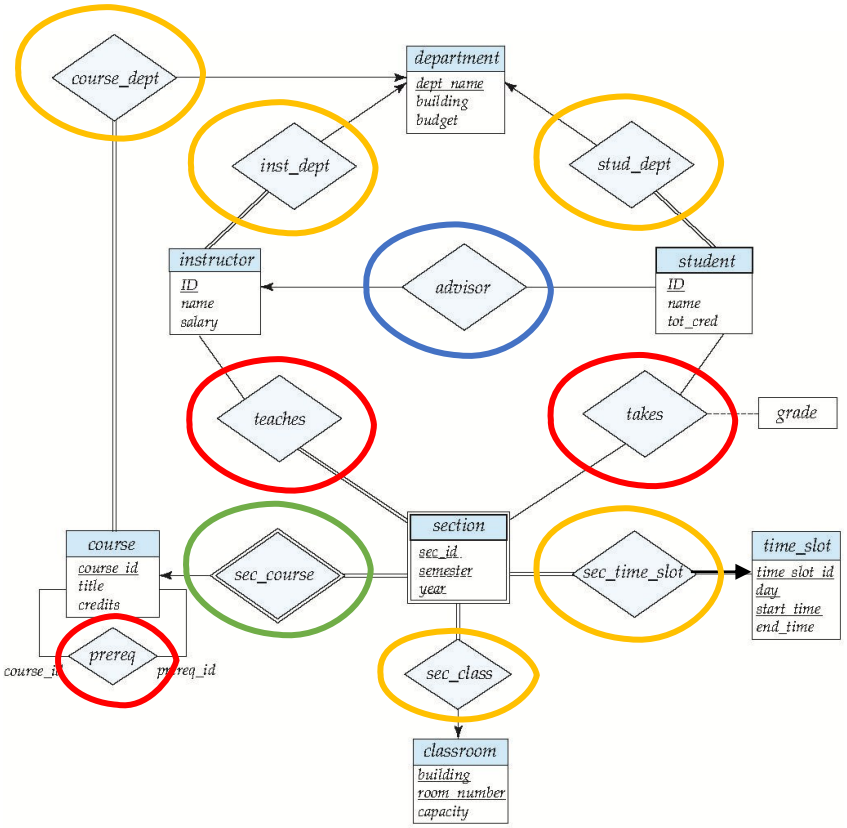
ER Diagram -> Relational Schema Diagram

Many-to-many

Many-to-one/one-to-many (partial)

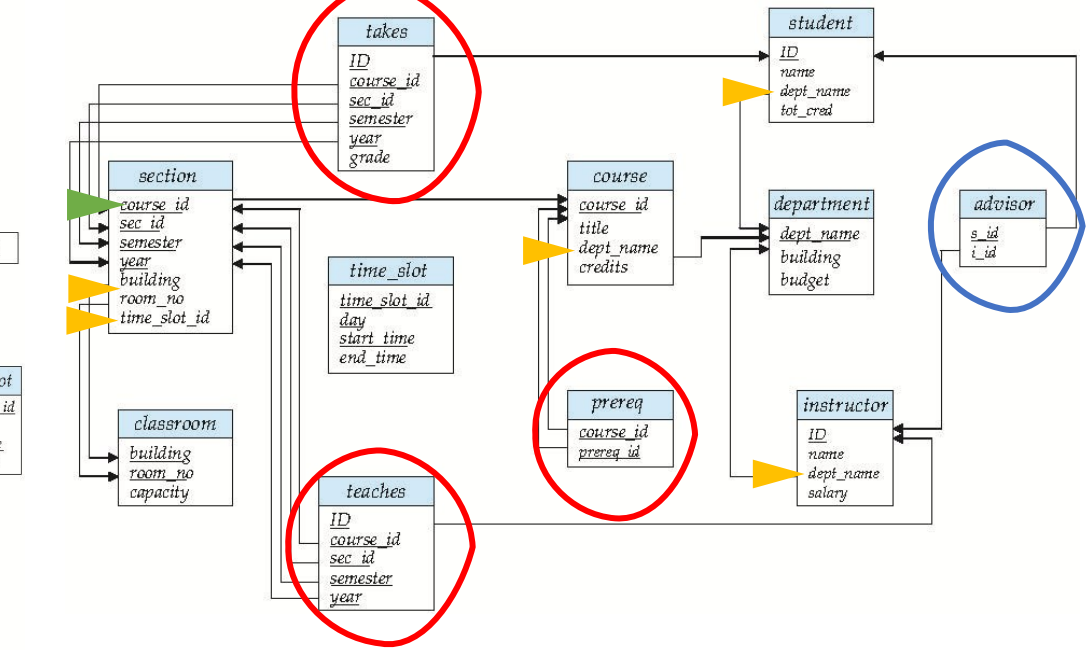
Many-to-one/one-to-many (total)

Many-to-one/one-to-many (weak)



ER Diagram

(7 entity sets & 10 relationship sets)

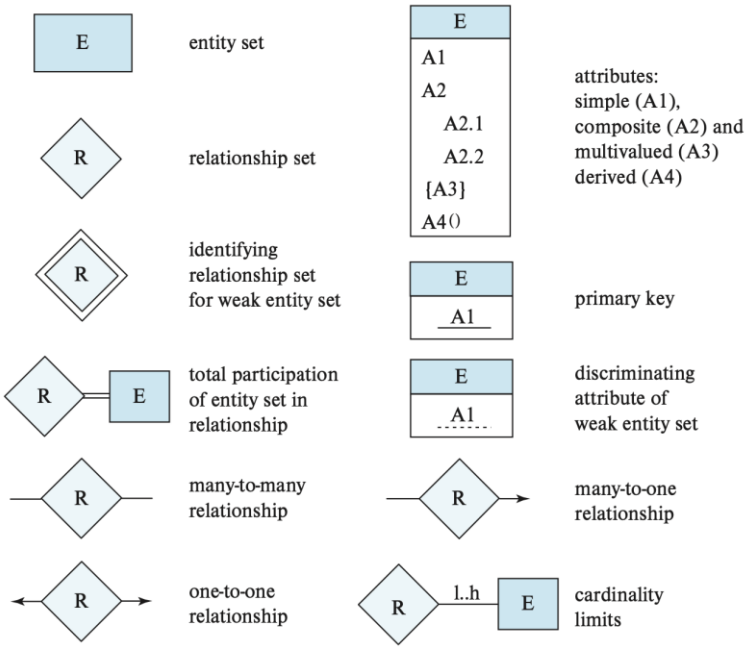


Relational Schema Diagram

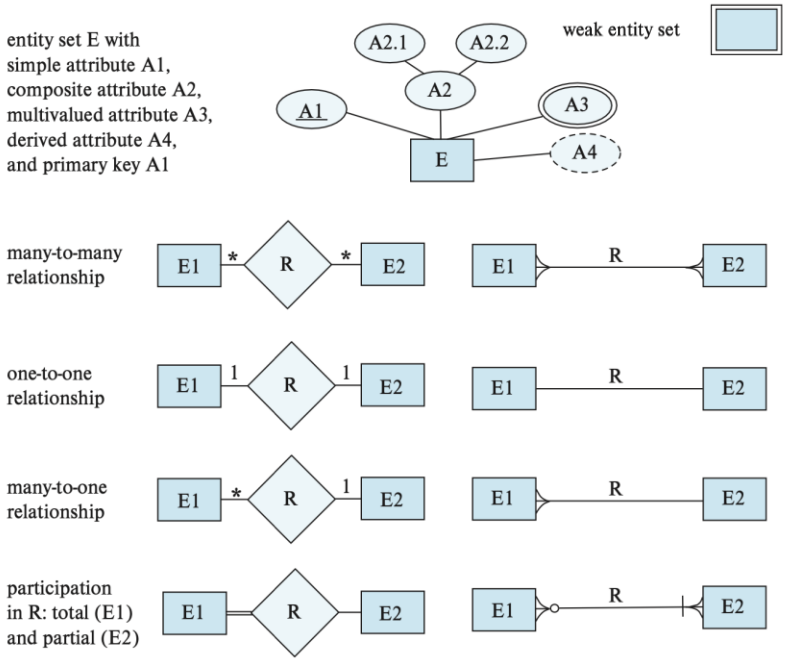
(11 tables)

Alternative ER Notations

- There is **no universal standard** for E-R diagram notation, and different books and E-R diagram software use different notations.



Notation used in db-book



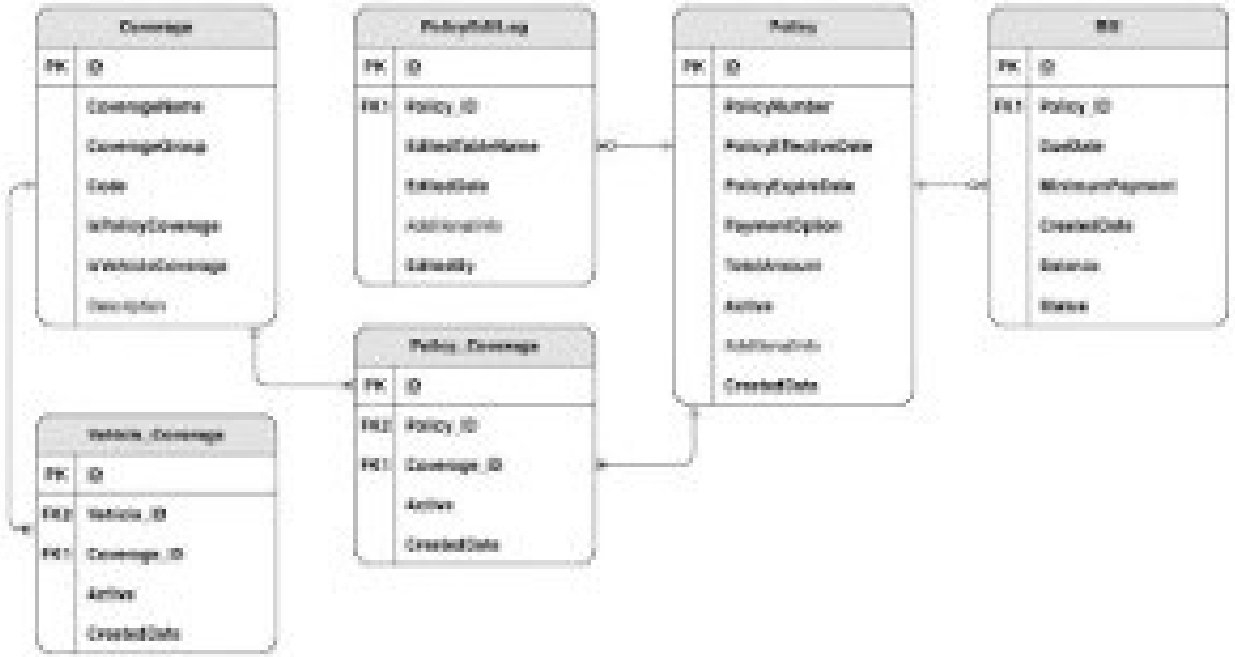
Chen's notation **Crow's Foot notation**

pgAdmin: Entity-Relationship Diagram (ERD) Tool Demo



https://www.youtube.com/watch?v=2pxVCzRFGeg&ab_channel=EDB

Creating ER/Schema Diagrams with *Draw.io*



https://www.youtube.com/watch?v=JYZPdU5F2iM&ab_channel=draw.io

Normalization

Discussion



- Is there any problem with this design?

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Source: http://www.db-book.com/

Discussion



ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Issues About This Design

- Redundancy of information
  - The building and budget of departments
- Complication of update
  - Changing the budget of a department needs to be reflected in multiple rows
- Inability to represent certain information
  - Inserting a new department without information of faculty members (ID, name, salary).



ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

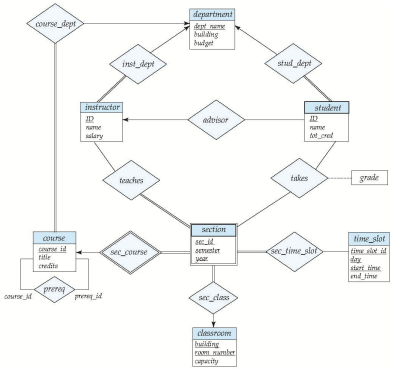


ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Source: http://www.db-book.com/

Database Approaches

- Need to come up with a methodology to ensure that each of the relations in the database is “good”.
- Two ways of doing so:
  - Entity relationship (E-R) data model: models an enterprise as a collection of entities and relationships.

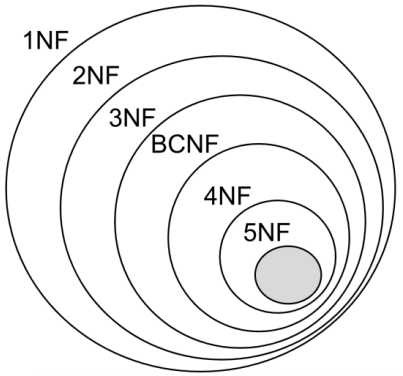


  - Normalization theory: the goal is to generate a set of relation schemas that allows us to store information without unnecessary redundancy, yet also allows us to retrieve information easily.

Source: http://www.db-book.com/

Normalization

- Normalization of a relational model is a process of analyzing the given relations based on their functional dependencies and candidate keys to minimize redundancy and insertion, deletion, and update anomalies.
- The normalization procedure entails various normal form tests which are typically sequentially evaluated.
- Unsatisfactory relations that do not meet the normal form tests are decomposed into smaller relations.

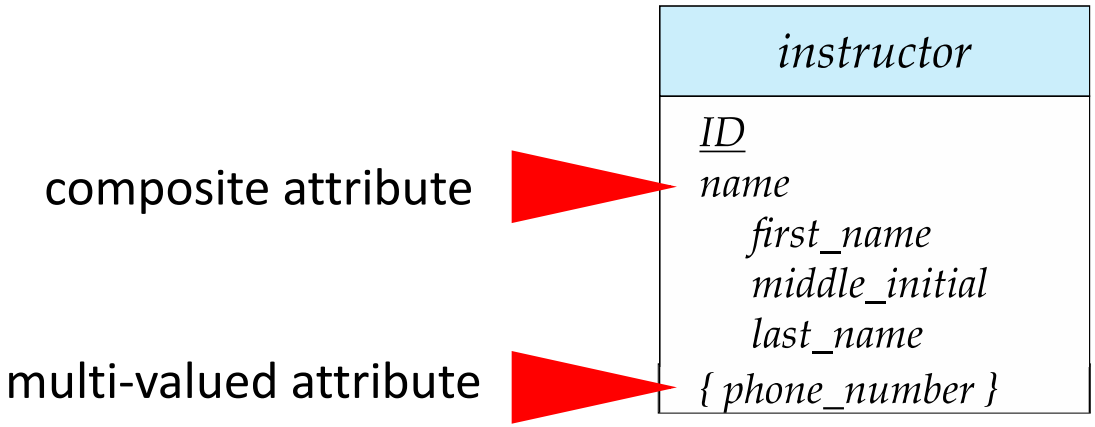


Functional Dependency

- A functional dependency X → Y between two sets of attribute types X and Y implies that a value of X uniquely determines a value of Y.
- For example,
  - student ID → student name
  - {building, room_number} → capacity
- We also say that there is a functional dependency from X to Y or that Y is functionally dependent on X.
- A functional dependency is a generalization of the notion of a *key*.

First Normal Form (1NF)

- **First Normal Form (1NF)**: every attribute type of a relation must be atomic (indivisible) and single-valued.
- Hence, no composite or multi-valued attribute types are tolerated.



instructor(ID, ***first_name, middle_initial, last_name****)*

inst_phone= (***ID, phone_number****)*

ID	phone_number
22222	456-7890
22222	123-4567

Source: http://www.db-book.com/

Second Normal Form (2NF)

- **Second Normal Form (2NF)**: In addition to meeting the requirements of 1NF, this form eliminates partial dependencies by ensuring that each non-key column is fully dependent on the primary key.
- Example: Consider a table *order_prod(order_id, product_id, product_name, quantity)*
  - *order_id, product_id* → *quantity* (full dependency)
  - *product_id* → *product_name* (partial dependency)

A functional dependency X → Y is a full functional dependency if removal of any attribute type A from X means that the dependency does not hold anymore.

- To achieve 2NF, we split the table into two tables:
  - *order(order_id, product_id, quantity)*
  - *product(product_id, product_name)*

Third Normal Form (3NF)

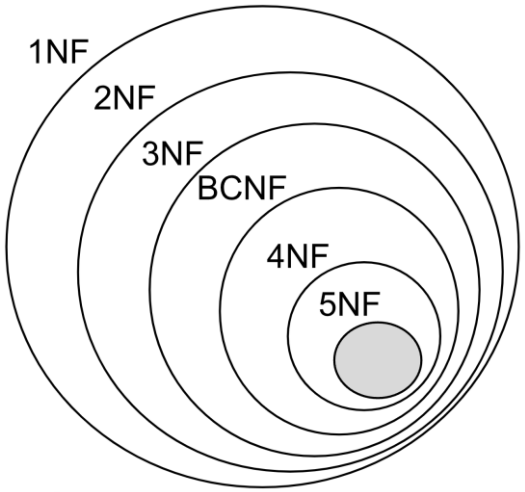
- **Third Normal Form (3NF)**: In addition to meeting the requirements of 2NF, this form eliminates transitive dependencies. It ensures that no non-key column is dependent on another non-key column.
- Example: Consider a table *inst_dept(inst_id, inst_name, salary, dept_name, building, budget)*
  - *inst_id* → *inst_name/salary* (full dependency)
  - *inst_id* → *dept_name* → *building/budget* (transitive dependency)

A functional dependency X → Y in a relation R is a transitive dependency if there is a set of attribute types Z that is neither a candidate key nor a subset of any key of R, and both X → Z and Z → Y hold.

- To achieve 3NF, we split the table into two tables:
  - *instructor(inst_id, inst_name, salary)*
  - *department(dept_name, building, budget)*

Higher Normal Forms

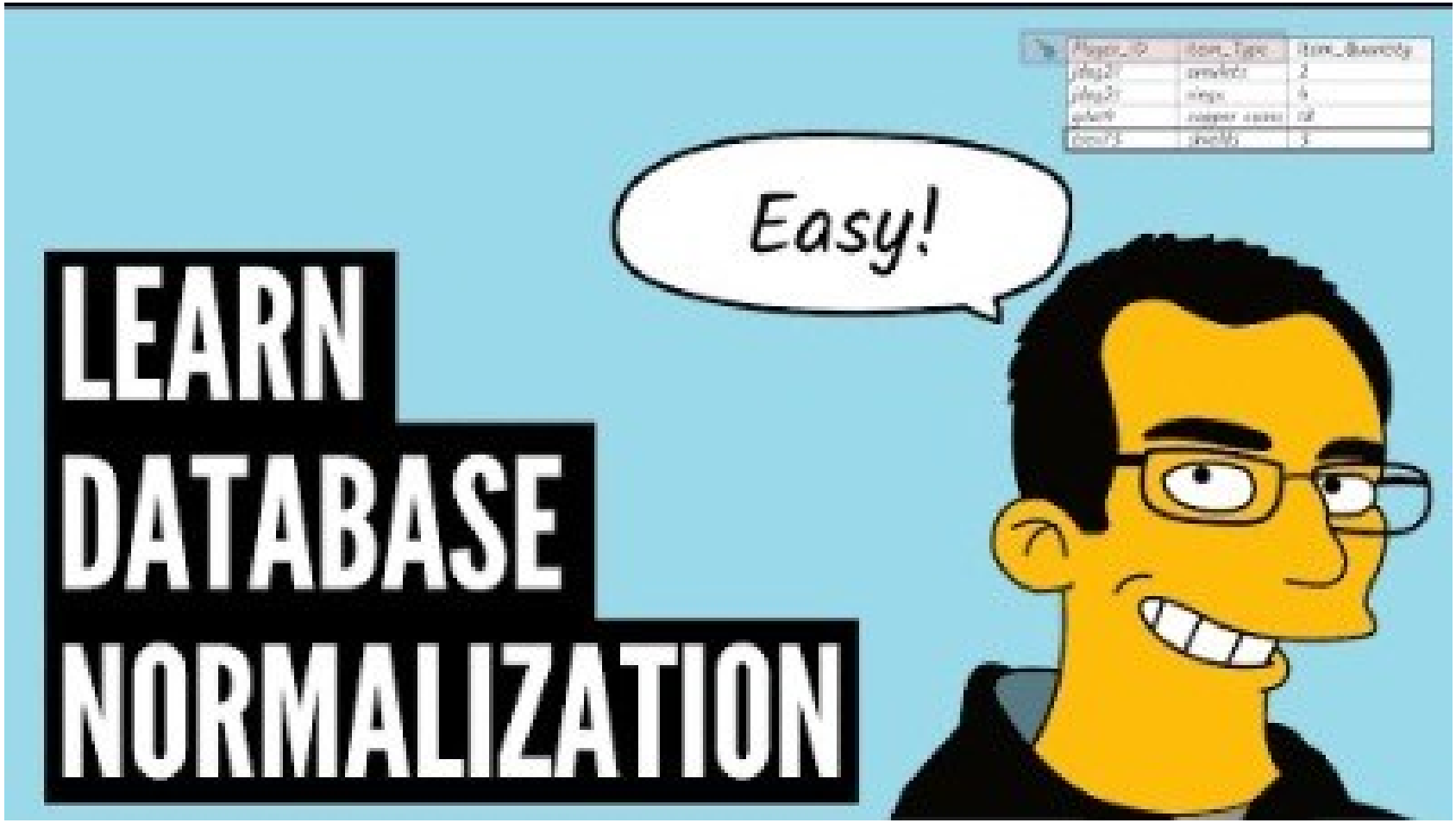
- There are also higher normal forms, such as Boyce-Codd Normal Form (BCNF) and Fourth Normal Form (4NF), that are employed to handle more complex data relationships and dependencies.



- The choice of which normal form to use depends on the specific requirements and complexity of the data being modeled.

Source: Google Image

Learn Database Normalization - 1NF, 2NF, 3NF, 4NF, 5NF



https://www.youtube.com/watch?v=GFQaEYEc8_8&ab_channel=Decomplexify

ER Model and Normalization

- When an E-R diagram is carefully designed, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization.
- However, in a real (imperfect) design, there can be functional dependencies from non-key attributes of an entity to other attributes of the entity
  - Example: an *instructor* entity with
  - attributes
    department_name and *building*,
  - functional dependency
    department_name $\rightarrow$ *building*
  - Good design would have made *department* an entity

Summary



- Understand the process of database design.
- Understand the usage of ***ER diagram***.
- Understand how to map ***ER diagrams*** to ***relation schemas***.
- Understand the concept of **normalization**.



Thank you!

michael-yue.yu@polyu.edu.hk

Source: Google Image